Data Mining Assignment 2: Recommender Systems

Simple Movie Recommendation Report

GitHub repository: https://github.com/alperendavran/assigment-2-alperen-davran-data-mining

1. Introduction

The goal of this assignment is to build and compare recommender models for movie recommendation, and then generate 10 recommendations for each user in `ratings_test.csv`. I focused on the two things that the assignment asks for most clearly:

- rating prediction quality, measured with `RMSE`
- top-10 recommendation quality, measured mainly with `Precision@10` and `Recall@10`

I compared a bias-based baseline, user-based collaborative filtering, and SVD matrix factorization, which matches what the assignment asks for. I also trained `NMF` in Surprise as a small extra matrix factorization experiment. The final system still had to handle cold-start users, because some users in the test file do not appear in the training data.

2. Data Understanding And Preprocessing

The training data contains `97,801` ratings from `600` users on `9,680` movies. The user-item matrix is about `98.32%` sparse, so most possible user-movie pairs are missing. This is important because sparse data usually makes neighborhood methods less stable and gives more room for latent-factor methods to help.

Some other statistics are also useful:

- average ratings per user: `163.0`, median: `69.5`
- average ratings per movie: `10.1`, median: `3.0`
- share of ratings from the top `10%` most-rated movies: `59.66%`
- movies with zero training ratings: `62`

These numbers show a clear long-tail structure. A small set of movies gets many ratings, while most movies receive only a few. Because of that, popular movies are easier to recover, so popularity-heavy recommendations can look stronger than expected.

During preprocessing, I checked missing values, duplicate user-movie ratings, and the rating range. No missing values and no duplicate ratings were found. I also added simple movie statistics such as average rating and rating count, which were later useful for analysis and for the cold-start fallback.

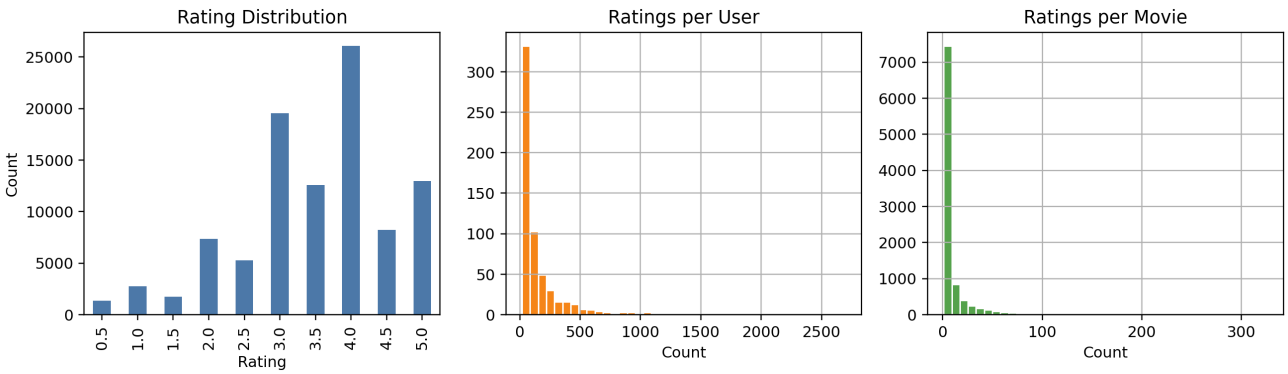

Figure 1: Basic data overview. The plots show rating concentration, uneven user activity, and a strong long-tail on the movie side.

Finally, I checked whether the users in `ratings_test.csv` were present in the training data. There were `90` warm users and `10` cold-start users, so `10%` of the final recommendations required a fallback strategy.

3. Models And Evaluation Setup

For validation, I used a temporal split. For each user, the ratings were sorted by timestamp and the last `5` interactions were held out. The remaining ratings were used for training. I chose this because the timestamps make it possible to respect interaction order instead of using a fully random split.

For ranking evaluation, only held-out ratings of `4.0` or higher were treated as relevant items. In this setup, `1,696` held-out ratings were relevant, and `533` out of `600` users had at least one relevant held-out item. This means `88.83%` of users contributed to the ranking evaluation.

The three models were:

Baseline. This model predicts ratings from the global mean together with user and item bias terms. It gives a simple reference point and helps separate real collaborative effects from global popularity and bias effects. In Surprise, this is `BaselineOnly`, which predicts from $\mu + b_u + b_i$ [3].

User-based collaborative filtering. This model estimates a user's preference from similar users. It is intuitive and easy to explain, but it depends strongly on overlap between users. In sparse data, that overlap can be weak.

SVD matrix factorization. This model learns latent user and item factors together with bias terms. It is a standard matrix factorization approach and is often more suitable for sparse recommender data because it does not rely as directly on local user-user overlap [1, 2].

NMF (extra experiment). Non-negative matrix factorization is another built-in option in Surprise. I used a biased variant with a small grid of regularization settings so that `RMSE` stayed comparable to `SVD`. I did *not* use it for the final recommendations; it is only there to check that a second factorization model does not automatically win on my split.

I reported `RMSE`, `Precision@10`, and `Recall@10` as the main metrics because they match the assignment most directly. I kept `HR@10` and `NDCG@10` as supporting metrics only.

I did not want to rely on `RMSE` alone. The task is to suggest movies, and a model can still get reasonable rating error while putting relevant items too low in the ranked list. That is why I always look at top-10 behavior together with `RMSE`.

The general idea behind top-N lists and precision/recall at K matches the explanations in the Surprise FAQ [4]. My validation setup uses a temporal holdout and sampled negatives, so it is not identical to the library examples, but the metric definitions are consistent with that reference.

Limitations. The ranking step ranks each relevant item against random unseen negatives, not against the full catalog, so the `Precision@10` and `Recall@10` numbers are mainly useful for comparing models on the same protocol. The cold-start fallback for new users is a popularity list, so it is limited by design.

4. Results And Discussion

The main validation results are shown in Table 1.

Table 1: Main validation results

Model	RMSE	P@10	R@10	HR@10	NDCG@10	Time
Baseline	0.9389	0.0756	0.2255	0.4503	0.1761	0.3
User-Based CF	0.9398	0.0370	0.1137	0.2927	0.0711	1.6
SVD	0.9366	0.0638	0.1894	0.4071	0.1424	0.4
NMF (extra)	0.9363	0.0101	0.0275	0.0807	0.0212	0.4

At first glance, the baseline looks surprisingly strong, especially on `Recall@10`. I think a likely explanation is the long-tail structure of the data. Since nearly `60%` of all ratings come from the top `10%` most-rated movies, popular items are easier to recover. A model that captures global movie effects can therefore perform well on ranking metrics even without much personalization.

To make this trade-off more visible, I also compared the recommendation lists themselves across `50` sampled users.

Table 2: Recommendation statistics over 50 sampled users

Model	Coverage	Avg. popularity	Max repeat	Avg. genres
Baseline	43	129.4	46	10.1
User-Based CF	252	14.1	15	10.0
SVD	160	88.1	17	11.6
NMF	156	9.9	33	8.7

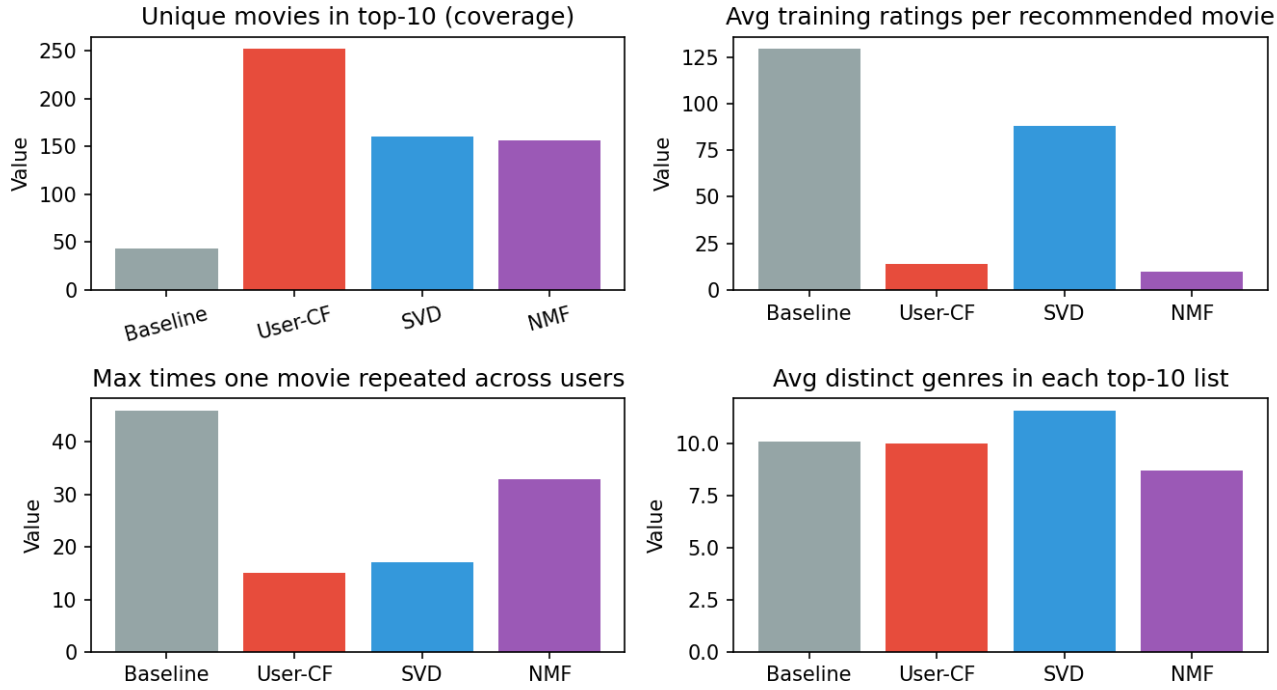

Figure 2: List-level statistics over 50 sampled users (distinct genres counted per top-10 list from `movies.csv` tags).

These results help explain the model behavior more clearly. Although the baseline was strong on some ranking metrics, its recommendation lists were much more concentrated and repeated the same popular items across users. Table 2 and Figure 2 make this concrete: low coverage and a very high max repeat for the baseline, compared with `SVD`.

The user-based model is much more personalized, but its accuracy is clearly weaker. `SVD` sits between the two on popularity and coverage. `NMF` reached a similar `RMSE` after tuning, but its top-10 metrics were much worse on the same validation setup, so it did not look like a better choice for recommendations here.

In this dataset, the user-based model suffers from the data structure itself: the matrix is about `98%` sparse and the median movie has only `3` ratings, so user-user overlap is often too limited for stable neighborhoods. I did not choose `SVD` because it won every metric: the baseline is still competitive on some top-10 scores. I chose it because it offered the most reasonable compromise between personalization and predictive quality on this dataset: clearly better ranking than user-based `CF`, and much less concentrated lists than the baseline, while keeping a strong `RMSE`.

I also tested a small hyperparameter search for SVD. However, it was optimized for `RMSE` on random folds and did not improve `Precision@10` or `Recall@10` on the temporal validation setup, so I did not use it as the final model.

5. Final Recommendation Strategy

For the final recommendation step, I retrained SVD on the full training set and generated the top `10` unseen movies for each warm user.

The final pipeline handled two cases:

- **Warm users:** score all unseen movies with SVD and return the top `10`

- **Cold-start users:** return a fallback list based on global popularity (weighted rating)

The fallback was needed for `10` out of `100` test users. For those users, collaborative filtering cannot personalize recommendations because there is no interaction history. Instead of using a plain average rating, I used a weighted rating formula so that movies with only a few ratings would not look artificially strong.

This is not a perfect solution to the cold-start problem, but it is simple, transparent, and reasonable for the scope of the assignment.

6. Conclusion

This dataset is challenging because it is highly sparse and strongly long-tail. Those properties make models that mainly follow global popularity look stronger than expected on some metrics, and they make user-based neighborhoods less stable.

For warm users, I did not pick `SVD` because it beat every number in Table 1. I picked it because it was the most reasonable compromise for me: more personalized lists than the baseline (as in Table 2), and more convincing ranking behavior than user-based `CF`, without pretending the problem is solved. The final system uses `SVD` for warm users and a weighted popularity fallback for cold-start users.

I do not claim that this fully solves recommendation quality or cold-start. However, I think it is a logical and well-motivated solution for the scope of the assignment.

As future work, a hybrid approach such as LightFM could combine collaborative signals with movie metadata (for example genres) [5]. That direction is appealing for cold-start items, but the official examples often emphasize implicit feedback, whereas this assignment also requires explicit rating error in the form of `RMSE`, so I kept the main experiments on Surprise-based models.

References

[1] Y. Koren, R. Bell, and C. Volinsky. Matrix factorization techniques for recommender systems. *Computer*, 42:30–37, 2009. https://www.chrisvolinsky.com/publications/17545-matrix-factorization-techniques-for-recommender-systems

[2] Surprise documentation. Matrix factorization-based algorithms. https://surprise.readthedocs.io/en/stable/matrix_factorization.html

[3] Surprise documentation. Basic algorithms. https://surprise.readthedocs.io/en/stable/basic_algorithms.html

[4] Surprise documentation. FAQ: top-N recommendations and precision/recall at K. https://surprise.readthedocs.io/en/stable/FAQ.html

[5] LightFM documentation. Hybrid matrix factorization models. https://making.lyst.com/lightfm/docs/home.html